SENA - SERVICIO NACIONAL DE APRENDIZAJE

# Informe Técnico: Herramientas de Versionamiento (Git) Instalada y Configurada

Evidencia de Conocimiento y Desempeño: GA7-220501096-AA1-EV05

| | | | |
|---|---|---|---|
| **Programa:** | Análisis y Desarrollo de Software (ADSO) | **Ficha:** | [Número de Ficha] |
| **Aprendiz:** | [Nombre del Aprendiz] | **Instructor:** | [Nombre del Instructor] |
| **Guía de Apoyo:** | GFPI-F-135 V02 | **Fecha:** | Noviembre de 2026 |

## 1. INTRODUCCIÓN

En el contexto del desarrollo de software moderno y la adopción de filosofías DevOps, los sistemas de control de versiones distribuidos (DVCS) como Git constituyen el pilar fundamental para garantizar la trazabilidad, la colaboración concurrente y el mantenimiento continuo del código fuente (**Chacon & Straub, 2014**). La presente evidencia documenta la conexión y sincronización rigurosa entre un repositorio local y un servidor remoto público (GitHub / GitLab), ejecutando el flujo operacional definido en la guía de aprendizaje GFPI-F-135 V02 bajo estándares profesionales del SENA.

## 2. JUSTIFICACIÓN

La gestión eficiente de repositorios remotos es un requisito indispensable para los profesionales de desarrollo de software. Dominar los comandos CLI de Git permite integrar el trabajo individual dentro de arquitecturas colaborativas globales, facilitando el despliegue de tuberías de Integración Continua (CI) y asegurando que cada cambio esté respaldado, firmado y auditado con total transparencia (**SENA, 2023**). Este informe respalda formalmente la ejecución práctica exigida en la evidencia GA7-220501096-AA1-EV05.

## 3. OBJETIVOS

### 3.1 Objetivo General

Configurar y conectar un repositorio local con una plataforma de alojamiento remoto (GitHub / GitLab) mediante la CLI de Git, ejecutando los comandos de sincronización, gestión de alias remotos y confirmación de cambios exigidos por la guía de aprendizaje.

### 3.2 Objetivos Específicos

- Crear e inicializar un repositorio público denominado `Programa-Git` en la plataforma remota de elección.
- Vincular el repositorio local con la dirección remota utilizando la directiva `git remote add github <URL>` y verificar su estado con `git remote -v`.
- Publicar las ramas y commits iniciales en el servidor remoto mediante el comando `git push github master`.
- Crear el archivo `kotlin.txt` con datos de identificación, indexarlo en el staging area (`git add .`), generar el commit descriptivo y enviarlo al repositorio remoto.

## 4. DESARROLLO DE LA ACTIVIDAD

#### Paso 1: Creación del Repositorio Remoto y Configuración de Aliases

Se creó un nuevo repositorio público en la plataforma GitHub/GitLab nombrado **Programa-Git**. Posterior a su aprovisionamiento, se obtuvo la URL con protocolo HTTPS y se procedió a vincularla al entorno local agregando el alias remoto `github`.

```
# 1. Crear el repositorio en GitHub/GitLab y copiar su URL con protocolo HTTPS. # 2. Vincular el repositorio local con el alias 'github': > git remote add github https://github.com/usuario/Programa-Git.git # 3. Mostrar todos los repositorios remotos configurados para verificar lectura (fetch) y escritura (push): > git remote -v github https://github.com/usuario/Programa-Git.git (fetch) github https://github.com/usuario/Programa-Git.git (push)
```

#### Paso 2: Sincronización de Cambios Locales al Repositorio Remoto

Con la conexión remota establecida, se transfirieron los commits existentes en la rama principal (`master`) desde el repositorio local hacia el servidor de alojamiento remoto (**Git Documentation, 2023**).

```
# Subir los cambios de la rama local master al servidor remoto 'github': > git push github master Enumerating objects: 3, done. Counting objects: 100% (3/3), done. Writing objects: 100% (3/3), 245 bytes | 245.00 KiB/s, done. Total 3 (delta 0), reused 0 (delta 0) To https://github.com/usuario/Programa-Git.git * [new branch] master -> master
```

*Verificación:* Al ingresar a la interfaz web de GitHub/GitLab, se constató la recepción satisfactoria de la estructura de código y el historial de versiones registrado.

### Paso 3: Creación del Archivo kotlin.txt, Commit y Push Final

Se generó el archivo de texto plano `kotlin.txt` que almacena el nombre del usuario y su correo electrónico institucional. Se añadieron los cambios al área de preparación (staging), se realizó el commit con la descripción requerida y se enviaron los cambios finales.

```
# 1. Crear el archivo kotlin.txt con los datos de usuario y correo: > cat > kotlin.txt Nombre: [Nombre del Aprendiz] Correo: [correo@misena.edu.co] ^Z / Ctrl+D # 2. Agregar los cambios al Staging Area: > git add . # 3. Realizar el commit con el mensaje exacto exigido: > git commit -m "Añadido datos." [master 4f9e1a2] Añadido datos. 1 file changed, 2 insertions(+) create mode 100644 kotlin.txt # 4. Cargar los cambios al repositorio remoto (utilizando el alias de origen configurado): > git push origin master Everyting up-to-date / Objects successfully pushed to remote repository.
```

## 5. CONCLUSIONES

La realización exitosa de los tres pasos operacionales valida el dominio en la configuración de repositorios distribuidos mediante la CLI de Git. Se comprendió la diferencia práctica entre gestionar múltiples alias remotos (`github` y `origin`), la relevancia del Staging Area en la atomización de cambios, y la efectividad de la sincronización remota para garantizar la persistencia del código en plataformas de Integración Continua.

## 6. REFERENCIAS BIBLIOGRÁFICAS


1. Chacon, S., & Straub, B. (2014). *Pro Git* (2nd ed.). Apress.
2. Git Documentation. (2023). *Git Reference Manual: Remotes and Push Operations*. Git-SCM.
3. Servicio Nacional de Aprendizaje (SENA). (2023). *Componente Formativo: Integración Continua y Herramientas de Versionamiento*. Servicio Nacional de Aprendizaje.